Ruby on Rails - An introduction

>>One of the most popular languages currently is quite easy to get into after all it puts the user ahead of everything else.>

Before we get into understanding what Ruby on Rails is we need to know a bit about Ruby, the user friendly language that has taken the world by storm.

A little about Ruby

Ruby is a programming language created by Yukihiro Matsumoto in the 90's. The first version was officially released in 1999. It's similar to C and Java as they are all general purpose programming languages. Moreover, it is one of the most popular programming languages out there and most of it is because of the popularity of Rails.

So what's this Rails thingy?

Simply put, Rails is a software library. It extends Ruby and adds a lot more functions allowing for rapid development. It was created by David Heinemeier Hansson. In Ruby, each package, or library that extends the language is called a Gem. Rails is one such RubyGem. Rails enables conventions for allowing collaboration when building websites. All of these conventions are put together as the Rails API which has been documented extensively and has immense popularity online. With a huge community behind you can be assured that it'll be around for quite a long time. A significant portion of the community is made up of startups. This community comes together to improve Rails collaborating on standards so that complex sites can be developed rapidly.

Going about using Rails

Starting off with Rails is more about learning the conventions noted in its API since that is all there is to it. Rails brings together HTML, CSS and Javascript to create web applications that run on a web server. And as such due to the requirement of a web server, this language is considered to be server-sided or back end application development platform.

Basic concepts

With all programming languages, there is an underlying philosophy which guides the growth and popularity of the said language. Rails is no different.

1. Opinionated

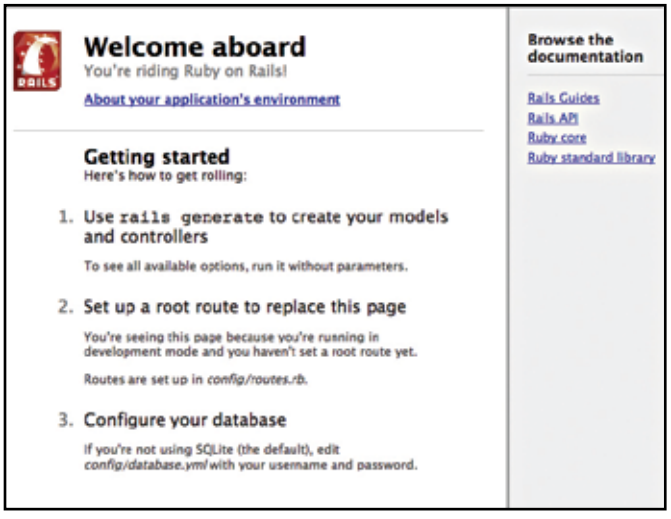
With programming languages there are often multiple ways of going about programming an application. A non-opinionated language is one that doesn't direct you to program in a certain fashion. Take Perl as an example, there is no right way or wrong way of going about getting to your objective. So there will be approaches that may not be as efficient as others. Rails on the other hand is opinionated. There is a right way about programming in Rails and this helps reduce the number of decisions one has to make along the way. Also, collaboration is made easier since everyone is following the same style of programming.

2. Convention over Configuration

This is simply why Ruby is described as opinionated. There are certain conventions that are followed strictly in Rails. In other languages if you were to create any method or function and describe all the variables, you'd have to specify each and every one of their data types and lengths. Ruby isn't completely devoid of this flexibility but it does make a few assumptions. If you were to create a model object in Rails and call it "User", then Rails will automatically assume it to be plural and save a database called "users" without the need for any extra configuration needed. This allows the programmer to spend more time on the main concept of an application rather than worry about the nitty gritty details which might throw a spanner in the gears.

3. Don't repeat yourself

Avoid duplicating your code. Not only does duplication make code more complex but it also makes it difficult to maintain and provides avenues for more bugs to creep into the code. This is not only limited to how your program but also to the development process as a whole. An example would be to use automated testing methodologies



The first page you see when you test out a new web application